HƯỚNG DẪN CÔNG VIỆC CHI TIẾT BTL2: SOCIAL GOLFER PROBLEM (SGP)

Định dạng đầu ra: Bài báo khoa học chuẩn Elsevier (`cas-sc`) | **Nhóm:** 3 thành viên (~33.3% mỗi người)
Nội dung so sánh: 6 Bộ SAT Encoders vs. Constraint Programming (CP) vs. Integer Linear Programming (ILP)
Công cụ bắt buộc: PySAT, Google OR-Tools CP-SAT, Gurobi/MIP, CNSPlots

---

TỔNG QUAN PHƯƠNG PHÁP & CÔNG CỤ CHỐT CHO ĐỀ TÀI

1. **6 Bộ mã hóa SAT:**
2. Sequential Counter Encoding (Module 3 / Sinz)
3. Commander Encoding (Module 3)
4. Binary Encoding (Module 3)
5. Product Encoding (Module 3)
6. Direct Encoding gốc ($x_{i,j,k}$)
7. Order Encoding (`Order Encoding.pdf`)
8. **2 Mô hình đối chứng chính xác:**
9. Constraint Programming (CP) giải bằng **Google OR-Tools CP-SAT**
10. Integer Linear Programming (ILP) giải bằng **Gurobi Optimizer** (hoặc OR-Tools MIP)
11. **Đa SAT Solvers:** CaDiCaL, Glucose, Kissat (qua `PySAT`)
12. **4 Kỹ thuật Phá vỡ đối xứng (Symmetry Breaking):** SB1 (Tuần 1), SB2 (Đầu nhóm), SB3 (Nội bộ nhóm), SB4 (Thứ tự tuần)
13. **Đồ họa khoa học:** Thư viện `cnsplots` (`https://cnsplots.farid.one/latest/`)
14. **Mẫu báo cáo:** Template LaTeX Elsevier `\documentclass[a4paper,fleqn]{cas-sc}` + Mục `Data Availability` dẫn link GitHub

---

BẢNG PHÂN CHIA NHIỆM VỤ TỔNG QUAN

Hạng mục	Thành viên 1 🔵	Thành viên 2 🟢	Thành viên 3 🟣
Đọc tài liệu	• `Assignment 2 - req.pdf` • `Mau bao cao 2DBMP.pdf` (Sec 1, 4) • `6 - SGP.pdf` (Sec 1, 2) • Docs OR-Tools CP & CNSPlots	• `Module 2 - SAT Encoding` • `Module 3 - AMO ALO` (Trang 5–16) • `6 - SGP.pdf` (Sec 3, 4.1, 4.2) • Docs PySAT (CardEnc & Solvers)	• `Order Encoding.pdf` (Sec III-B) • `6 - SGP.pdf` (Sec 2.2, 4.4) • `Mau bao cao 2DBMP.pdf` (Sec 2.1) • `Module 4 - SAT Solvers`
Cần Code những gì	• `src/io_utils.py` (CLI & DIMACS) • `src/verifier.py` (Universal Verifier) • `src/display.py` (In ma trận lịch) • `src/cp_solver.py` (OR-Tools CP) • `benchmarks/plot_cns.py` (CNSPlots)	• `src/sat_direct_encoders.py` (Ánh xạ biến, C1, C2 với 4 bộ AMO: SeqCounter, Commander, Binary, Product; C3 Social) • `src/sat_runner.py` (Tích hợp CaDiCaL, Glucose, Kissat)	• `src/order_encoder.py` (Order Encoding) • `src/ilp_solver.py` (Mô hình Gurobi/MIP) • `src/symmetry.py` (4 kỹ thuật SB1–SB4) • `benchmarks/run_all.py` (Benchmark suite)
Viết Bài báo Elsevier	• Abstract & Title • Section 1: Introduction • Section 2: CP Formulation • Section 4: Computational Experiments • Section 5: Conclusion & Data Avail.	• Section 3.1: Boolean Direct Modeling • Section 3.2: 4 Kỹ thuật AMO & Cardinality (SeqCounter, Commander, Binary, Product; bảng số biến/clauses)	• Section 2: ILP Formulation • Section 3.3: Order Encoding • Section 3.4: Symmetry Breaking (SB1–SB4)
Đầu ra khác	Quản lý toàn bộ template LaTeX `cas-sc`	Quản lý kho mã nguồn GitHub, viết `requirements.txt` và `README.md`	Thiết kế Slide thuyết trình (15–20 trang), chuẩn bị kịch bản Demo
TỶ LỆ KHỐI LƯỢNG	**33.3%**	**33.3%**	**33.4%**

HƯỚNG DẪN CHI TIẾT CHO TỪNG THÀNH VIÊN

🥷 THÀNH VIÊN 1 (TV1): Lead I/O, Solution Verifier, Mô hình CP, Vẽ CNSPlots & Báo cáo Elsevier

1. Đọc & Tìm hiểu những tài liệu nào?

1. `Assignment 2 - requirements.pdf` **(Trang 1 và 2):**
2. Nắm rõ yêu cầu so sánh SAT với CP (CP-SAT), yêu cầu bắt buộc dùng thư viện `cnsplots` để vẽ đồ thị, template LaTeX Elsevier `cas-sc`, và mục dẫn link GitHub `Data Availability`.
3. `6 - Social Golfer Problem - SGP.pdf` **(Section 1 và Section 2, Trang 1–4):**
4. Bài báo của Thầy (TS. Tô Văn Khánh) về SGP: Hiểu định nghĩa bài toán SGP(w, g, s), tổng số người $n = g \cdot s$, cận trên số tuần lý thuyết $w_{max} \le \lfloor \frac{gs-1}{s-1} \rfloor$, và các điều kiện nghiệm hợp lệ.

5. `Mau xay dung bao cao BTL so 1 v BTL so 2.pdf` **(Section 1 và Section 4):**
6. Xem cách các thầy viết Introduction chuẩn quốc tế và cách thiết kế bảng kết quả, biểu đồ so sánh với solver thương mại.
7. **Tài liệu Google OR-Tools CP-SAT (Python) & Thư viện `cnsplots`:**
8. Cách tạo biến nguyên, dùng hàm `AddAllDifferent()`, đo thời gian giải.
9. Cách dùng `cnsplots` vẽ Cactus plot và Line plot cho bài báo khoa học.

2. Cần Code những gì?

- `src/io_utils.py`:
- Viết bộ phân tích dòng lệnh (CLI parser) nhận các cờ: số tuần `-w`, số nhóm `-g`, số người mỗi nhóm `-s`, phương pháp `--method` (direct/order/cp/ilp), bộ AMO `--amo` (seqcounter/commander/binary/product), solver `--solver` (cadical/glucose/kissat), cấu hình `--sb` (none/sb1/full).
- Viết hàm kiểm tra cận trên lý thuyết: nếu $w > \lfloor \frac{gs-1}{s-1} \rfloor$, báo ngay vô nghiệm (UNSAT) mà không cần chạy solver.
- Viết hàm xuất danh sách mệnh đề ra file text chuẩn định dạng DIMACS `.cnf`.
- `src/display.py`:
- Viết hàm định dạng và in bảng lịch thi đấu ma trận theo từng tuần, từng nhóm rõ ràng, dễ nhìn.
- `src/verifier.py` **(Universal Solution Verifier):**
- Viết thuật toán kiểm tra tính đúng đắn độc lập của lời giải thu được từ **bất kỳ solver nào** (SAT, CP hay ILP) qua 3 luật:
  1. *Mỗi tuần:* Mỗi người chơi đúng 1 lần (không ai vắng mặt, không ai chơi 2 lần).
  2. *Mỗi nhóm:* Đủ đúng s người chơi.
  3. *Không gặp lại:* Mọi cặp người chơi gặp nhau trong cùng nhóm không quá 1 lần trong suốt w tuần.
- `src/cp_solver.py` **(Mô hình Constraint Programming):**
- Sử dụng Google OR-Tools CP-SAT (`ortools.sat.python.cp_model`):
  - Đặt biến số nguyên đại diện cho người chơi trong từng nhóm.
  - Áp dụng ràng buộc toàn cục `AddAllDifferent` cho mỗi tuần.
  - Áp dụng ràng buộc Social (số người trùng nhau giữa 2 nhóm ở 2 tuần khác nhau ≤ 1).
  - Giải bài toán, trích xuất nghiệm và đo thời gian giải (CPU time).
- `benchmarks/plot_cns.py` **(Vẽ đồ thị CNSPlots):**
- Đọc file dữ liệu `results.csv` do TV3 chạy ra, dùng thư viện `cnsplots` vẽ:
  - **Cactus Plot:** So sánh số bài toán giải được theo thời gian giữa SAT (Cadical), CP (CP-SAT), và ILP (Gurobi).
  - **Runtime Line Plot:** So sánh thời gian chạy của 4 phương pháp AMO khi số người n tăng dần.
  - **Bar Plot:** So sánh hiệu năng Direct Encoding vs. Order Encoding.

3. Viết những phần nào trong Bài báo Elsevier (`cas-sc`)?

- **Abstract & Title:** Tóm tắt mục tiêu, phương pháp (6 SAT Encoders vs CP vs ILP), và các kết quả cốt lõi đạt được.
- **Section 1: Introduction:** Nêu bối cảnh bài toán SGP, ứng dụng thực tế (xếp lịch hội thảo, thi đấu), cấu trúc bài báo và tóm tắt đóng góp.
- **Section 2: Background (Phần CP):** Mô tả toán học mô hình Constraint Programming và sức mạnh của ràng buộc toàn cục `AllDifferent`.
- **Section 4: Computational Experiments:** Lập bảng số liệu kết quả, chèn các hình vẽ từ `cnsplots`, phân tích so sánh đối đầu giữa SAT, CP và ILP.

- **Section 5: Conclusion & Data Availability:** Tóm tắt kết luận, đánh giá hạn chế, nêu hướng phát triển và dẫn link GitHub chứa mã nguồn.

🥷 THÀNH VIÊN 2 (TV2): Kỹ sư 4 Bộ AMO Encoders, Direct Modeling & Đa SAT Solvers

1. Đọc & Tìm hiểu những tài liệu nào?

1. `Module2_SAT Encoding.pdf` **(Trang 1–27):**
2. Nắm chắc nguyên lý Direct Encoding (One-hot), cách ánh xạ biến Boolean 3 chiều thành số nguyên dương 1-based (DIMACS).
3. `Module3_AMO ALO Encoding.pdf` **(Trọng tâm - Trang 5–16):**
4. *Trang 5–9:* Nguyên lý **Binary Encoding** (dùng các biến bitwise).
5. *Trang 10–11:* Nguyên lý **Sequential Counter Encoding** (chuỗi biến phụ mô phỏng bộ đếm).
6. *Trang 12–13:* Nguyên lý **Commander Encoding** (chia cụm biến kèm biến chỉ huy).
7. *Trang 14–16:* Nguyên lý **Product Encoding** (chiếu biến lên ma trận lưới 2 chiều).
8. `6 - Social Golfer Problem - SGP.pdf` **(Section 3, Section 4.1 & 4.2):**
9. Xem cách thầy định nghĩa biến Boolean 3 chỉ số $x_{i,j,k}$, công thức mệnh đề ràng buộc Social C3, và kỹ thuật New Sequential Counter (NSC) cho ràng buộc kích thước nhóm.
10. **Tài liệu thư viện `PySAT`:**
11. Module `pysat.card.CardEnc` và các enum `EncType.seqcounter`, `EncType.commander`, `EncType.bitwise`, `EncType.product`.
12. Module `pysat.solvers`: Cách gọi `Cadical153`, `Glucose4`, `Kissat`.

2. Cần Code những gì?

- `src/sat_direct_encoders.py` **(Mã hóa Direct & 4 Bộ AMO):**
- Viết hàm ánh xạ 2 chiều giữa bộ chỉ số (i, j, k) và số nguyên `var_id` theo công thức:
  $$\text{ID}(i, j, k) = (i - 1) \cdot g \cdot w + (j - 1) \cdot w + k$$
- Viết hàm sinh mệnh đề cho Ràng buộc C1 (Mỗi người đúng 1 nhóm/tuần: ALO + AMO).
- Viết hàm sinh mệnh đề cho Ràng buộc C2 (Mỗi nhóm đúng s người - Exactly-s), hỗ trợ lựa chọn **4 kỹ thuật AMO/Cardinality**:
  1. `seqcounter`: Dùng `pysat.card.CardEnc.equals(..., encoding=EncType.seqcounter)`
  2. `commander`: Dùng `pysat.card.CardEnc.equals(..., encoding=EncType.commander)`
  3. `binary`: Dùng `pysat.card.CardEnc.equals(..., encoding=EncType.bitwise)`
  4. `product`: Dùng `pysat.card.CardEnc.equals(..., encoding=EncType.product)`
- Viết hàm sinh mệnh đề cho Ràng buộc C3 (Social constraint: Không có 2 người chơi cùng nhóm quá 1 lần) thành các mệnh đề 4 literals:
  $$[\neg x_{i_1, j_1, k_1}, \neg x_{i_2, j_1, k_1}, \neg x_{i_1, j_2, k_2}, \neg x_{i_2, j_2, k_2}]$$
- `src/sat_runner.py` **(Bộ chạy Đa SAT Solvers):**
- Viết module nhận danh sách mệnh đề CNF và gọi lần lượt các solver qua PySAT: **CaDiCaL**, **Glucose**, **Kissat**.
- Thu thập các thông số kỹ thuật: Trạng thái SAT/UNSAT, thời gian chạy thực tế (CPU time), số biến, số mệnh đề sinh ra, số lần xung đột (conflicts).
- **Quản lý kho mã nguồn GitHub:**
- Tạo repository Git, cấu trúc các thư mục chuẩn.

- Viết file `requirements.txt` và viết file `README.md` hướng dẫn chi tiết cách cài đặt môi trường và các lệnh chạy thử nghiệm.

3. Viết những phần nào trong Bài báo Elsevier (`cas-sc`)?

- **Section 3.1: Boolean Direct Modeling:**
- Định nghĩa hình thức hệ thống biến Boolean $x_{i,j,k}$.
- Công thức hóa toán học cho 3 ràng buộc gốc C1, C2, C3 sang dạng CNF.
- **Section 3.2: AMO and Cardinality Encodings:**
- Trình bày cơ sở lý thuyết của 4 phương pháp: Sequential Counter, Commander, Binary, Product.
- Lập bảng so sánh lý thuyết về độ phức tạp: Số biến chính, số biến phụ phát sinh, và số lượng mệnh đề theo n, g, s, w.

🥷 THÀNH VIÊN 3 (TV3): Chuyên gia Order Encoding, Mô hình ILP, Symmetry Breaking & Benchmark

1. Đọc & Tìm hiểu những tài liệu nào?

1. `Order Encoding.pdf` **(Trọng tâm - Section III-B, Trang 5–6):**
2. Bài báo của Thầy (TS. Tô Văn Khánh) về Order Encoding: Nắm vững nguyên lý biến trật tự: Biến nguyên $X \in [1..D]$ biểu diễn qua chuỗi biến Boolean $B_{X \le v}$. Mối quan hệ tự nhiên: $X \le v \to X \le v+1$.
3. `6 - Social Golfer Problem - SGP.pdf` **(Section 2.2 & Section 4.4, Trang 4, 11–13):**
4. Nắm rõ phân tích không gian đối xứng của SGP có $(n!)^w$ cấu hình tương đương.
5. Hiểu rõ 4 kỹ thuật phá vỡ đối xứng tĩnh: **SB1 (Cố định tuần 1)**, **SB2 (Thứ tự nhóm)**, **SB3 (Nội bộ nhóm)**, **SB4 (Thứ tự tuần)**.
6. `Mau xay dung bao cao BTL so 1 v BTL so 2.pdf` **(Section 2.1, Trang 4–5):**
7. Nắm cách xây dựng mô hình Quy hoạch tuyến tính nguyên (ILP) dạng chuẩn toán học: biến quyết định, hàm mục tiêu, hệ ràng buộc tuyến tính.
8. `Module4_Design of SAT solver.pdf` **(Trang 15–20):**
9. Hiểu cơ chế học mệnh đề từ xung đột (CDCL) để giải thích lý do tại sao không gian đối xứng làm chậm solver và tại sao SB giúp tăng tốc.

2. Cần Code những gì?

- `src/order_encoder.py` **(Mã hóa Order Encoding):**
- Xây dựng mô hình biến nguyên: $P_{w, g, c} \in [1 \dots n]$ là golfer thứ c của nhóm g tại tuần w.
- Định nghĩa biến Boolean trật tự: $B_{w, g, c, v} \iff (P_{w, g, c} \le v)$ với $v \in [1 \dots n-1]$.
- Sinh các mệnh đề trật tự cơ sở: $[\neg B_{w, g, c, v}, B_{w, g, c, v+1}]$.
- Sinh các mệnh đề ép thứ tự trong nhóm: $P_{w, g, c} < P_{w, g, c+1} \implies [\neg B_{w, g, c, v}, B_{w, g, c+1, v}]$ (vừa mã hóa vừa tự động phá vỡ đối xứng nội bộ nhóm).
- Sinh các mệnh đề cấm trùng cặp qua các tuần.
- `src/ilp_solver.py` **(Mô hình Quy hoạch tuyến tính nguyên ILP):**
- Mô hình hóa SGP bằng Gurobi (`gurobipy`) hoặc OR-Tools Linear Solver:
  - Biến nhị phân: $x_{i,j,k} \in {0, 1}$.
  - Ràng buộc: Mỗi người đúng 1 nhóm/tuần; Mỗi nhóm đúng s người.
  - Tuyến tính hóa điều kiện gặp nhau bằng biến phụ $y_{i_1, i_2, j, k} \ge x_{i_1, j, k} + x_{i_2, j, k} - 1$ và $\sum_w \sum_j y \le 1$.

    - Giải bài toán và ghi nhận thời gian thực thi của ILP solver.

- `src/symmetry.py` **(4 Kỹ thuật Symmetry Breaking):**
- Cài đặt 4 hàm độc lập có cờ bật/tắt:
    1. `SB1` : Cố định tuần 1 (gán cứng golfer i vào nhóm $\lceil i/s \rceil$ bằng Unit Clauses).
    2. `SB2` : Cố định Golfer 1 luôn ở Nhóm 1; sắp xếp thứ tự golfer nhỏ nhất giữa các nhóm.
    3. `SB3` : Sắp xếp các chỉ số golfer trong cùng nhóm tăng dần theo ID.
    4. `SB4` : Cố định thứ tự xuất hiện của bạn cùng nhóm với Golfer 1 qua các tuần tiếp theo.
- `benchmarks/run_all.py` **(Benchmark Suite tự động):**
- Viết script tự động quét qua danh sách test cases $SGP(2,2,2) \to SGP(8,8,8)$.
- Đo ma trận kết quả: 6 Encoders $\times$ 3 Solvers (SAT, CP, ILP) $\times$ Cấu hình SB.
- Xuất dữ liệu sạch ra file `benchmarks/results.csv` cho TV1.

3. Viết những phần nào trong Bài báo Elsevier (`cas-sc`) & Slide?

- **Section 2: Background (Phần ILP Formulation):** Mô tả toán học mô hình ILP cho SGP và kỹ thuật tuyến tính hóa tích biến nhị phân qua biến phụ.
- **Section 3.3: Order Encoding for SGP:** Mô tả nguyên lý Order Encoding và giải thích ưu thế duy trì tính lan truyền suy diễn (Unit Propagation) trên quan hệ thứ tự.
- **Section 3.4: Symmetry Breaking Constraints:** Phân tích toán học 4 nhóm đối xứng (tuần, nhóm, người chơi) và viết công thức cụ thể của SB1, SB2, SB3, SB4.
- **Slide thuyết trình (15–20 trang):** Thiết kế slide theo chuẩn hội nghị khoa học và chuẩn bị terminal sẵn sàng demo trực tiếp tốc độ giải của các solver.

---

4. TIẾN ĐỘ THỰC HIỆN 6 TUẦN

- **Tuần 1:** Cả nhóm đọc đúng các phần tài liệu được giao. Họp cuối tuần 1 để chốt hiểu biết lý thuyết và tạo Git repo.
- **Tuần 2:** Thiết kế công thức toán học trên giấy. TV1 nháp Section 1; TV2 lập bảng biến và công thức 4 bộ AMO; TV3 lập công thức Order Encoding và mô hình ILP.
- **Tuần 3 (Sprint Code 1):** TV1 code `io_utils.py` , `verifier.py` ; TV2 code `sat_direct_encoders.py` (4 bộ AMO); TV3 code `symmetry.py` , `ilp_solver.py` .
- **Tuần 4 (Sprint Code 2):** TV1 code `cp_solver.py` ; TV2 hoàn thiện `sat_runner.py` ; TV3 code `order_encoder.py` và `run_all.py` . **Mốc M3: Chạy thành công SGP(2,2,2) và SGP(3,3,3) trên cả SAT, CP, ILP!**
- **Tuần 5 (Thực nghiệm):** TV3 chạy `run_all.py` xuất `results.csv` ; TV1 dùng `cnsplots` vẽ toàn bộ biểu đồ; TV2 tối ưu code và viết docstrings.
- **Tuần 6 (Báo cáo & Thuyết trình):** Cả nhóm ráp bài báo LaTeX `cas-sc` ; TV2 hoàn thiện README GitHub; TV3 hoàn thiện Slide và kịch bản Demo; tập dượt thuyết trình và nộp bài.